04-system-design.md system-design[1]

ProjectEnhancer — System Design

Document type: Detailed system design
Status: v1.0
Audience: Product, UX, backend, worker, security, and QA teams

1. System boundaries

Client application

Provides GitHub sign-in, repository picker, single-viewport wizard, stack confirmation, health tabs, repair review, recommendation pagination, diff/preview, approvals, and Version 2 status.

Application API

Owns sessions, projects, runs, permissions, recommendations, approvals, manifests, and signed worker-job requests. It never executes arbitrary repository code in the web process.

Worker plane

Runs repository inspection and validation in isolated, time-limited workers. Workers receive a snapshot and a narrow job definition, then return structured evidence and safe summaries.

Storage

Stores project metadata, immutable commit references, redacted logs, findings, approvals, manifests, validation receipts, and short-lived preview artifacts.

2. Data model

```
User
  -> Project
      -> RepositorySnapshot
          -> Run
              -> Finding
              -> Recommendation
              -> RepairItem
              -> Approval
              -> ValidationReceipt
              -> ChangeManifest
```

Project

```
id, github_owner, repository, default_branch, access_scope, created_at
```

Snapshot

```
id, project_id, commit_sha, branch, captured_at, file_manifest_hash
```

Finding

```
id, run_id, category, status, severity, summary, evidence_refs, affected_paths
```

Recommendation

```
id, run_id, category, risk, scope, summary, affected_paths,
affected_symbols, behavior_impact, approval_state
```

Change manifest

```
run_id, base_commit, target_branch, added, modified, moved, deleted,
approved_recommendations, validation_id, rollback_ref
```

3. State machine

```
REPOSITORY_SELECTED
  -> DETECTING
  -> AWAITING_STACK_CONFIRMATION
  -> COMPLETENESS_REVIEW
  -> AWAITING_SCOPE
  -> BASELINE_TESTING
  -> REPAIR_REQUIRED | BASELINE_HEALTHY
  -> REPAIR_APPROVAL
  -> REPAIR_VALIDATING
  -> BASELINE_HEALTHY
  -> ENHANCEMENT_ANALYZING
  -> RECOMMENDATION_REVIEW
  -> VERSION_2_BUILDING
  -> VERSION_2_VALIDATING
  -> READY_FOR_PR | VALIDATION_FAILED | CANCELLED
```

No state transition may skip stack confirmation or completeness/scope selection. Version 2 cannot be marked ready when validation failed.

4. Baseline check strategy

The adapter discovers safe commands from repository configuration, then runs available checks in an isolated environment:

```
dependencies -> build -> lint/typecheck -> unit -> integration
-> functional/system/runtime -> security
```

The orchestrator records command availability before execution. A missing command is **Not available**; a non-zero result is **Failed**. A partial project excludes not-implemented areas instead of reporting them as failures.

5. Repair strategy

1. Create a repair workspace from the immutable snapshot.
2. Generate repair items only for pre-existing, evidenced defects.
3. Require user approval per item or approved group.
4. Apply the smallest safe change.
5. Re-run affected and baseline checks.

6. Store a repair diff separately from enhancement recommendations.
7. Block enhancement if repair validation is still unstable, unless the user explicitly selects Existing Parts Only and the affected area is excluded.

6. Enhancement strategy

The analyzer builds a project map from files, symbols, routes, UI elements, dependencies, tests, and runtime evidence. It produces recommendations without editing. The change worker applies only approved recommendations and writes a manifest for every file operation.

For file moves or renames, the system must list updated imports and references. For UI changes, it must identify component and element names and provide before/after evidence when a preview environment is available.

7. Security design

- OAuth uses least-privilege repository scopes.
- Tokens are encrypted, short-lived where possible, and never logged.
- Workers run with restricted filesystem, network, CPU, memory, and time.
- Secrets are detected and redacted from logs and model prompts.
- User projects are isolated from other projects and workers.
- All sensitive actions require an authorization check and an audit event.
- Pull request creation is separate from automatic merge.

8. Responsive UI architecture

Routes represent major phases: `/home`, `/project/select`, `/project/detect`, `/project/confirm`, `/health`, `/repair`, `/enhancement`, `/recommendations/:id`, and `/version-2`. Tabs handle related checks. Pagination handles repository lists, issues, and recommendations. At mobile breakpoints, the split preview becomes sequential panels with explicit Next/Back navigation.

9. Failure handling

Every worker job is resumable or safely cancellable. Timeouts produce a clear “could not complete” state, not a false pass. If a change creates new failures, the target branch is paused and the original project remains unaffected. The user can inspect logs, retry, discard the branch, or return to the last validated checkpoint.

10. Verification checklist

- Confirm the original base commit is unchanged.
- Confirm all applied changes have approvals.
- Confirm added/moved/deleted files match the manifest.
- Confirm available checks were run and receipts are stored.
- Confirm no critical security issue is hidden by a pass state.
- Confirm unknown technologies are not guessed.
- Confirm incomplete areas are excluded or explicitly scoped.
- Confirm desktop, tablet, and mobile screens obey the single-viewport rule.
- Confirm a “Duck” review can compare implementation against all four PDFs.